Fast Vector Quantization Algorithm for ScaNN

Yasuhiro Fujiwara
NTT, Inc.
Kanagawa, Japan
yasuhiro.fujiwara@ntt.com

Ángel López García-Arias
NTT, Inc.
Kanagawa, Japan
angel.lopez@ntt.com

Yasutoshi Ida
NTT, Inc.
Tokyo, Japan
yasutoshi.ida@ntt.com

Atsutoshi Kumagai
NTT, Inc.
Tokyo, Japan
atsutoshi.kumagai@ntt.com

Masahiro Nakano
NTT, Inc.
Kanagawa, Japan
ma.nakano@ntt.com

Makoto Nakatsuji
NTT, Inc.
Kanagawa, Japan
makoto.nakatsuji@ntt.com

Akisato Kimura
NTT, Inc.
Kanagawa, Japan
akisato@ieee.org

Abstract

Maximum Inner Product Search (MIPS) is a popular task to find the vector with the highest inner product for a given query. ScaNN is a score-aware quantization approach for MIPS that effectively transforms vectors with higher inner products into short sequences of codewords within codebooks. When quantizing vectors, it iteratively updates codebooks by assigning vectors to codewords and computing inverse matrices obtained from the assigned vectors. ScaNN, however, incurs a high computation cost when quantizing large-scale data. This is because (1) it computes quantization losses for all pairs of vectors and codewords, and (2) the size of the inverse matrices is quadratic in the number of dimensions. Our proposal, *F-ScaNN*, increases the efficiency of ScaNN through two techniques: (1) it computes the upper and lower bounds of the losses to assign vectors, and (2) it employs the conjugate gradient method to avoid computing the inverse matrix. Theoretically, we can obtain the same quantization results as ScaNN. Furthermore, we can improve search accuracy by using scaled codewords. Experiments show that our approach is significantly faster than previous approaches.

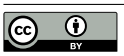
CCS Concepts

• **Information systems** → **Data structures**.

Keywords

Vector quantization, Efficient, Algorithm

ACM Reference Format:
Yasuhiro Fujiwara, Ángel López García-Arias, Yasutoshi Ida, Atsutoshi Kumagai, Masahiro Nakano, Makoto Nakatsuji, and Akisato Kimura. 2026. Fast Vector Quantization Algorithm for ScaNN. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780233



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780233

1 Introduction

The rapid growth of database systems has triggered a significant challenge in efficiently processing large-scale vector data [3, 4, 7]. Maximum Inner Product Search (MIPS) is a popular task for high-dimensional data that identifies the vector with the highest inner product for a given query [1, 14]. The naive approach to MIPS, computing inner products between the query and each vector, requires $O(nd)$ time if n and d are the numbers of vectors and dimensions, respectively. However, this cost becomes prohibitive if the database has large-scale data of high dimensionality.

Approximation approaches have been proposed for MIPS, including hashing [14], graph search [10, 13, 17], and partitioning [1]. These approaches prune vectors to compute inner products; they focus on effectively generating candidates for MIPS. Vector quantization, on the other hand, enables fast inner product computation for each vector [5, 8]; it focuses on computational acceleration. Since it is orthogonal to the pruning approaches, we can use vector quantization along with the pruning approaches to realize successful MIPS. Vector quantization represents a d-dimensional vector as a sequence of m codewords using m codebooks. Inner products are then efficiently computed using lookup tables that store precomputed inner products between codewords. If each codebook contains c codewords, quantizing a query takes $O(cd)$ time. As a result, it performs MIPS in $O(cd + nm)$ time since it needs $O(m)$ time to compute an inner product between the query and a vector. This computational cost is lower than the $O(nd)$ cost required by the naive approach if c and m are set appropriately.

Several approaches to vector quantization have been proposed, such as product quantization [8] and optimized product quantization [5]. They reduce residual error when quantizing vectors, where each vector contributes equally to computing the error. However, since MIPS finds the vector with the highest inner product, vectors with higher inner products should be quantized more accurately than others. ScaNN is proposed based on this observation [6]. It uses the score-aware quantization loss that more greatly penalizes error parallel to a vector than error orthogonal to a vector, since a vector with a large norm is expected to have the highest inner

product. Owing to its effectiveness, ScaNN is used in life sciences to process large-scale data, such as in genomic analysis [12].

However, ScaNN has a high computational cost when quantizing a large number of vectors. To obtain the codebooks, it performs the following iterative computations until convergence: it assigns each vector to the codeword with minimum score-aware quantization loss, and it computes each codeword using an inverse matrix obtained from the assigned vectors to minimize the total loss of the assigned vectors. However, vector assignment is computationally intensive since it necessitates computing the losses for all pairs of vectors and codewords. Furthermore, obtaining codewords is costly since the size of the inverse matrix grows quadratically with the number of dimensions. This paper proposes *F-ScaNN*, a fast quantization method for ScaNN. We safely prune unlikely codewords in assigning vectors to efficiently obtain the codebooks using the upper and lower bounds of the quantization loss. In addition, to avoid the inverse matrix computation, we use the conjugate gradient method [2] to obtain the codewords exactly that minimize the total loss of the assigned vectors. Since our approach theoretically obtains the same codebooks as the original approach, it does not degrade the quantization results to improve efficiency. Furthermore, we propose an approach that uses scaled codewords to improve search accuracy. In summary, the main contributions are as follows:

- Our approach computes codebooks through codeword pruning and the conjugate gradient method to improve efficiency.
- Our approach guarantees the same quantization results as the original approach. Moreover, the search accuracy of our approach can be improved using scaled codewords.
- Experiments confirm that the proposed approach is faster than previous approaches, with more accurate MIPS results.

The remainder of this paper is organized as follows: Section 2 describes related work. Section 3 overviews the background. Section 4 introduces our approach. Section 5 presents the experiments performed. Section 6 concludes the paper.

2 Related Work

Vector quantization decreases the processing time of the inner product computation itself by using codewords. Product quantization splits each d-dimensional vector into m subspaces and applies K-means to each separated subspace to derive codebooks [8]. Optimized product quantization improves product quantization by rotating vectors in the d-dimensional space [5]. These approaches focus on minimizing residual error by treating vectors equally. ScaNN is categorized as score-aware quantization that gives larger importance to vectors with higher inner products [6]. It has been extended in two different ways. Anisotropic additive quantization extends the score-aware quantization loss for additive quantization, where each codebook covers all features [16]. Specifically, it modifies the loss function of additive quantization by incorporating the parallel and orthogonal errors. Query-aware quantization estimates the query distribution in the d-dimensional space using a held-out set [15]. It computes the quantization loss from the estimated query distribution and uses an alternative optimization technique to iteratively update the codebooks. It efficiently samples vectors using K-means to obtain codewords.

Table 1: Main symbols.

Symbol	Definition
n	Number of vectors
d	Number of dimensions
c	Number of codewords
$p(\mathbf{q}, \mathbf{x}_i)$	Inner product of $\mathbf{q}$ and $\mathbf{x}_i$
$l(\mathbf{x}_i, \mathbf{w}_j)$	Quantization loss of $\mathbf{x}_i$ to $\mathbf{w}_j$
$h_{i,\parallel}$	Parallel weight of $\mathbf{x}_i$
$h_{i,\perp}$	Orthogonal weight of $\mathbf{x}_i$
$\mathbf{x}_i$	i-th vector
$\mathbf{w}_j$	j-th codeword

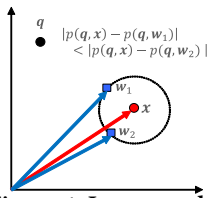


Figure 1: Inner product approximation error.

3 Preliminaries

This section provides an overview of ScaNN. Table 1 lists the main symbols. To simplify the presentation, we assume that a single codebook is computed from d-dimensional features. However, we can compute m codebooks by splitting each vector into m subspaces of d/m dimensions [5, 6, 8]. Given dataset $\mathbb{X}$ with n vectors, where each vector $\mathbf{x}_i = [x_i[1], \ldots, x_i[d]]^\top$ $(i = 1, 2, \ldots, n)$ lies within a d-dimensional space, MIPS finds vector $\mathbf{x}^*$ having the highest inner product with query $\mathbf{q} = [q[1], \ldots, q[d]]^\top$. Specifically, it finds

$$\mathbf{x}^* = \text{argmax}_{\mathbf{x}_i \in \mathbb{X}}\, p(\mathbf{q}, \mathbf{x}_i), \tag{1}$$

where $p(\mathbf{q}, \mathbf{x}_i)$ is the inner product of $\mathbf{q}$ and $\mathbf{x}_i$ (i.e., $p(\mathbf{q}, \mathbf{x}_i) = \mathbf{q}^\top \mathbf{x}_i$).

If $\mathbf{w}_j = [w_j[1], \ldots, w_j[d]]^\top$ is the j-th codeword $(j = 1, \ldots, c)$ and $\mathbb{W} = \{\mathbf{w}_1, \ldots, \mathbf{w}_c\}$ is a codebook, vector quantization replaces vector $\mathbf{x}_i$ with one of the codewords. Since MIPS is a task that finds the vector with the highest inner product, vectors expected to maximize the inner product should be quantized more accurately than others. ScaNN is a vector quantization based on the observation shown in Figure 1. In this figure, codewords $\mathbf{w}_1$ and $\mathbf{w}_2$ are located at the same distance from vector $\mathbf{x}$. However, $\mathbf{w}_1$ provides a better approximation of the inner product $p(\mathbf{q}, \mathbf{x})$ than $\mathbf{w}_2$, i.e., $|p(\mathbf{q}, \mathbf{x}) - p(\mathbf{q}, \mathbf{w}_1)| < |p(\mathbf{q}, \mathbf{x}) - p(\mathbf{q}, \mathbf{w}_2)|$. This is because $\mathbf{w}_1$ better preserves the norm of $\mathbf{x}$ than $\mathbf{w}_2$. As a result, ScaNN penalizes quantization error in the direction parallel to a vector more heavily than error in the orthogonal direction.

In ScaNN, let $\mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)$ be the residual error of vector $\mathbf{x}_i$ when $\mathbf{x}_i$ is quantized to $\tilde{\mathbf{x}}_i$, defined as $\mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = \mathbf{x}_i - \tilde{\mathbf{x}}_i$, it uses the following score-aware quantization loss:

$$l(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = \mathbb{E}_{\mathbf{q} \sim Q}[\omega(p(\mathbf{q}, \mathbf{x}_i))(p(\mathbf{q}, \mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)))^2], \tag{2}$$

where $\mathbb{E}_{\mathbf{q} \sim Q}[\cdot]$ is the expectation concerning query distribution Q. In Equation (2), $\omega : \mathcal{R} \mapsto \mathcal{R}^+$ is a monotonically non-decreasing and non-constant weight function that determines the importance of each vector concerning the quantization loss. Specifically, for threshold T, it sets $\omega(p(\mathbf{q}, \mathbf{x}_i)) = 1$ if $p(\mathbf{q}, \mathbf{x}_i) \geq T$, and $\omega(p(\mathbf{q}, \mathbf{x}_i)) = 0$ otherwise. This indicates that only vectors whose expected inner product is at least T contribute to the loss, while the others are ignored. The norm of q does not influence the ranking result of vectors in MIPS, and there is generally no prior knowledge of the query distribution Q. Therefore, ScaNN assumes that queries are unit vectors (i.e., $\|q\| = 1$) and are uniformly distributed in the sphere. The loss function of Equation (2) is designed to more effectively preserve the norm of $\mathbf{x}_i$ rather than its angle with $\mathbf{q}$. This is because a vector with a larger norm than others is expected to maximize the inner product, even if its angle with a query is large. Residual error $\mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = \mathbf{x}_i - \tilde{\mathbf{x}}_i$ of Equation (2) can be decomposed into parallel and orthogonal errors to the vector. If $\mathbf{r}_\parallel(\mathbf{x}_i, \tilde{\mathbf{x}}_i)$ is the

parallel residual error to vector $\mathbf{x}_i$, it is computed as

$$\mathbf{r}_{\parallel}(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = \frac{p(\mathbf{x}_i, \mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i))}{\|\mathbf{x}_i\|^2}\mathbf{x}_i. \quad (3)$$

Similarly, orthogonal residual error $\mathbf{r}_{\perp}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)$ is given as follows:

$$\mathbf{r}_{\perp}(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = \mathbf{r}(\mathbf{x}_i, \tilde{\mathbf{x}}_i) - \mathbf{r}_{\parallel}(\mathbf{x}_i, \tilde{\mathbf{x}}_i). \quad (4)$$

If queries are uniformly distributed in the d-dimensional sphere, score-aware quantization loss $l(\mathbf{x}_i, \tilde{\mathbf{x}}_i)$ can be computed as follows:

$$l(\mathbf{x}_i, \tilde{\mathbf{x}}_i) = h_{i,\parallel}\|\mathbf{r}_{\parallel}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)\|^2 + h_{i,\perp}\|\mathbf{r}_{\perp}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)\|^2, \quad (5)$$

where $h_{i,\parallel}$ and $h_{i,\perp}$ are the parallel and orthogonal weights for vector $\mathbf{x}_i$, respectively. The weights are given as follows:

$$h_{i,\parallel} = (d-1)\int_0^{\varphi}(\sin^{d-2}\theta - \sin^d\theta)d\theta, \quad h_{i,\perp} = \int_0^{\varphi}\sin^d\theta d\theta, \quad (6)$$

where $\varphi = \arccos(T/\|\mathbf{x}_i\|)$. These weights can be computed in $O(d^2)$ time using the following equation [6]:

$$\int_0^{\varphi}\sin^d\theta d\theta = \frac{d-1}{d}\int_0^{\varphi}\sin^{d-2}\theta d\theta - \frac{1}{d}\cos\varphi\sin^{d-1}\varphi. \quad (7)$$

ScaNN computes c codewords by solving the following optimization problem:

$$\min_{\mathbf{w}_1,\ldots,\mathbf{w}_c}\sum_{\mathbf{x}_i}\min_{\tilde{\mathbf{x}}_i\in\mathbb{W}}(h_{i,\parallel}\|\mathbf{r}_{\parallel}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)\|^2 + h_{i,\perp}\|\mathbf{r}_{\perp}(\mathbf{x}_i, \tilde{\mathbf{x}}_i)\|^2). \quad (8)$$

This problem is solved using an iterative algorithm analogous to K-means. In K-means, initial cluster centers are randomly selected from the data. Then, each point is assigned to the nearest center, and the centers are updated using the assigned points. These assignment and update steps are repeated until convergence. Similarly, ScaNN begins by initializing the codewords through random sampling. In each iteration, it assigns each vector to the codeword that minimizes the score-aware quantization loss. Then, each codeword is updated based on the vectors assigned to it. These two steps are repeated until the codewords converge.

ScaNN assigns each vector $\mathbf{x}_i$ to the codeword that minimizes the score-aware quantization loss as follows:

$$\tilde{\mathbf{x}}_i = \mathrm{argmin}_{\mathbf{w}_j\in\mathbb{W}}\, l(\mathbf{x}_i, \mathbf{w}_j). \quad (9)$$

Since it takes $O(d)$ time to compute $\mathbf{r}_{\parallel}(\mathbf{x}_i, \mathbf{w}_j)$ and $\mathbf{r}_{\perp}(\mathbf{x}_i, \mathbf{w}_j)$ for given $h_{i,\parallel}$ and $h_{i,\perp}$ from Equation (3) and (4), it requires $O(d)$ time to compute loss $l(\mathbf{x}_i, \mathbf{w}_j)$ from Equation (5). Therefore, the second step takes $O(cd)$ time to determine the assignment of $\mathbf{x}_i$ by computing the losses for the c codewords.

ScaNN computes codeword $\mathbf{w}_j$ as follows if $\mathbb{X}_j$ is the set of vectors assigned to $\mathbf{w}_j$:

$$\mathbf{w}_j = \mathbf{M}^{-1}\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp}\mathbf{x}_i. \quad (10)$$

In this equation, $\mathbf{M}$ is a $d \times d$ matrix given as follows:

$$\mathbf{M} = \mathbf{I}\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp} + \sum_{\mathbf{x}_i\in\mathbb{X}_j}\frac{h_{i,\parallel} - h_{i,\perp}}{\|\mathbf{x}_i\|^2}\mathbf{x}_i\mathbf{x}_i^{\top}, \quad (11)$$

where $\mathbf{I}$ is the identity matrix. If n_j is the number of vectors included in $\mathbb{X}_j$, it takes $O(n_j d^2)$ time to compute $\mathbf{M}$. Since it needs to compute inverse matrix $\mathbf{M}^{-1}$, Equation (10) takes $O(n_j d^2 + d^3)$ time to compute codeword $\mathbf{w}_j$.

ScaNN incurs high processing time when quantizing large-scale data. The computational cost to obtain the weights for n vectors is $O(nd^2)$. It needs $O(c)$ time to sample c vectors to initialize codewords. Additionally, each iteration requires $O(ncd)$ time to assign n vectors to codewords and $O(nd^2 + cd^3)$ time to compute c codewords since $\sum_{j=1}^{c} n_j = n$ holds. Consequently, if t is the number of iterations, the computational cost of ScaNN is $O(dt(nc + nd + cd^2))$. Furthermore, it requires $O(nd)$, $O(cd)$, $O(n)$, and $O(d^2)$ space to hold vectors, codewords, the weights, and $\mathbf{M}$, respectively. Therefore, the memory cost of ScaNN is $O(d(n + c + d))$.

4 Proposed Method

This section shows our approach to computing codewords efficiently. Proofs of lemmas and theorems are shown in the appendix.

4.1 Efficient Vector Assignment

As shown in Equation (9), each vector is assigned to the codeword that minimizes the loss. The original approach identifies the codeword of the minimum loss by computing the exact loss for all codewords, which results in a high computational cost due to evaluating every vector-codeword pair. To reduce this cost, we prune unlikely codewords during the assignment step by leveraging upper and lower bounds on the loss. Let $\overline{l}(\mathbf{x}, \mathbf{w}_j)$ and $\underline{l}(\mathbf{x}, \mathbf{w}_j)$ denote the upper and lower bounds of $l(\mathbf{x}, \mathbf{w}_j)$, respectively, such that $\overline{l}(\mathbf{x}, \mathbf{w}_j) \geq l(\mathbf{x}, \mathbf{w}_j)$ and $\underline{l}(\mathbf{x}, \mathbf{w}_j) \leq l(\mathbf{x}, \mathbf{w}_j)$. If we have $\overline{l}(\mathbf{x}, \mathbf{w}_j) < \underline{l}(\mathbf{x}, \mathbf{w}_k)$ for codewords $\mathbf{w}_j$ and $\mathbf{w}_k$, then it follows that $l(\mathbf{x}, \mathbf{w}_j) < l(\mathbf{x}, \mathbf{w}_k)$, indicating that $\mathbf{w}_k$ cannot yield the minimum loss for $\mathbf{x}$. Consequently, the exact loss computation for $\mathbf{w}_k$ can be safely skipped. In this section, we describe our approach to improving the efficiency of the vector assignment based on the upper and lower bounds. We obtain these bounds by computing the upper and lower bounds of the inner products between vectors and codewords.

4.1.1 Upper and Lower Bounds of the Inner Product. To compute the upper and lower bounds of the inner products in each iteration, we approximate vectors and codewords using Singular Value Decomposition (SVD) since it yields the smallest approximate error [11]. To approximate vector $\mathbf{x}_i = [x_i[1], \ldots, x_i[d]]^{\top}$ and codeword $\mathbf{w}_j = [w_j[1], \ldots, w_j[d]]^{\top}$ of d dimensions, we perform SVD of small rank (e.g., $\log d$) by sampling a small number of vectors (e.g., $\log n$). Specifically, let $\mathbf{S}$ be a $d \times \log n$ matrix formed by concatenating randomly sampled $\log n$ vectors, we compute the SVD of rank $\log d$ on matrix $\mathbf{S}$ as $\mathbf{U}\Sigma\mathbf{V}^{\top}$. We compute approximation $\mathbf{x}_i' = [x_i'[1], \ldots, x_i'[\log d]]^{\top}$ and $\mathbf{w}_j' = [w_j'[1], \ldots, w_j'[\log d]]^{\top}$ as

$$\mathbf{x}_i' = \mathbf{U}^{\top}\mathbf{x}_i, \;\; \mathbf{w}_j' = \mathbf{U}^{\top}\mathbf{w}_j. \quad (12)$$

Using $\mathbf{x}_i'$ and $\mathbf{w}_j'$, we compute the upper and lower bounds of the inner product $p(\mathbf{x}_i, \mathbf{w}_j)$ as follows:

Definition 4.1. Let $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ be the upper and lower bounds of $p(\mathbf{x}_i, \mathbf{w}_j)$, respectively. If $\dot{x}_i = \|\mathbf{x}_i\|^2 - \|\mathbf{x}_i'\|^2$ and $\dot{w}_j = \|\mathbf{w}_j\|^2 - \|\mathbf{w}_j'\|^2$ are squared residual norms of $\mathbf{x}_i$ and $\mathbf{w}_j$, respectively, the upper and lower bounds are given as follows:

$$\overline{p}(\mathbf{x}_i, \mathbf{w}_j) = p(\mathbf{x}_i', \mathbf{w}_j') + \sqrt{\dot{x}_i\dot{w}_j}, \;\; \underline{p}(\mathbf{x}_i, \mathbf{w}_j) = p(\mathbf{x}_i', \mathbf{w}_j') - \sqrt{\dot{x}_i\dot{w}_j}. \quad (13)$$

The following lemma holds for the upper and lower bounds:

Lemma 4.2. *We have the following upper and lower bounding property for $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$:*

$$\underline{p}(\mathbf{x}_i, \mathbf{w}_j) \leq p(\mathbf{x}_i, \mathbf{w}_j) \leq \overline{p}(\mathbf{x}_i, \mathbf{w}_j). \quad (14)$$

Note that we can compute $\mathbf{x}_i'$ and $\dot{x}_i$ before entering the iterations to solve the optimization problem. On the other hand, we need to

compute $\mathbf{w}'_j$ and $\dot{w}_j$ ad hoc in each iteration since codewords are updated in the iterations. The computational cost of the upper and lower bounds is given as follows:

Lemma 4.3. *Each iteration takes $O(c(n+d)\log d)$ time to compute the upper and lower bounds of inner products for the pairs of n vectors and c codewords.*

Lemma 4.2 and 4.3 indicate that we can efficiently compute bound $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ for $p(\mathbf{x}_i, \mathbf{w}_j)$, enabling us to efficiently compute the upper and lower bounds of the score-aware quantization loss, as described in the next section.

4.1.2 Upper and Lower Bounds of the Loss. Our approach prunes unlikely codewords when assigning vectors by using the following upper and lower bounds of the score-aware quantization loss:

Definition 4.4. Let $L_{i,j}(\lambda)$ be a function defined as follows:

$$L_{i,j}(\lambda) = \frac{h_{i,\parallel} - h_{i,\perp}}{\|\mathbf{x}_i\|^2}\lambda^2 - 2h_{i,\parallel}\lambda + h_{i,\parallel}\|\mathbf{x}_i\|^2 + h_{i,\perp}\|\mathbf{w}_j\|^2. \quad (15)$$

Upper bound $\overline{l}(\mathbf{x}_i, \mathbf{w}_j)$ and lower bound $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$ are as follows:

$$\overline{l}(\mathbf{x}_i,\mathbf{w}_j) = \begin{cases} L_{i,j}(\overline{p}(\mathbf{x}_i,\mathbf{w}_j)) & \text{if } \frac{1}{2}(\overline{p}(\mathbf{x}_i,\mathbf{w}_j) + \underline{p}(\mathbf{x}_i,\mathbf{w}_j)) \geq \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel} - h_{i,\perp}} \\ L_{i,j}(\underline{p}(\mathbf{x}_i,\mathbf{w}_j)) & \text{otherwise,} \end{cases}$$

$$\underline{l}(\mathbf{x}_i,\mathbf{w}_j) = \begin{cases} L_{i,j}(\overline{p}(\mathbf{x}_i,\mathbf{w}_j)) & \text{if } \overline{p}(\mathbf{x}_i,\mathbf{w}_j) < \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel} - h_{i,\perp}} \\ L_{i,j}(\underline{p}(\mathbf{x}_i,\mathbf{w}_j)) & \text{if } \underline{p}(\mathbf{x}_i,\mathbf{w}_j) > \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel} - h_{i,\perp}} \\ L_{i,j}\left(\frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel} - h_{i,\perp}}\right) & \text{otherwise.} \end{cases} \quad (16)$$

Note that we can compute $\overline{l}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$ in $O(1)$ time from Equation (16) after computing $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$. Based on a downward convex quadratic property of function $L_{i,j}(\lambda)$ for λ, we have the following property for $\overline{l}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$:

Lemma 4.5. *The following equation holds for $\overline{l}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$:*

$$\underline{l}(\mathbf{x}_i, \mathbf{w}_j) \leq l(\mathbf{x}_i, \mathbf{w}_j) \leq \overline{l}(\mathbf{x}_i, \mathbf{w}_j). \quad (17)$$

The proposed approach uses the upper and lower bounds of $l(\mathbf{x}_i, \mathbf{w}_j)$ to prune unlikely codewords when performing the vector assignment, as shown in the next section.

4.1.3 Assignment Algorithm. Algorithm 1 depicts the algorithm for efficiently assigning vector $\mathbf{x}_i$ to the codeword. In this algorithm, $\mathbf{w}$ represents the codeword that minimizes the loss for $\mathbf{x}_i$, and $\mathbb{C}$ denotes the set of candidate codewords that can minimize the loss. As explained in Section 4.1.1, ad hoc computations of $\mathbf{w}'_j$ and $\dot{w}_j$ are required since codewords are updated iteratively. Thus, $\mathbf{w}'_j$ and $\dot{w}_j$ are computed for each codeword (lines 1-3). To perform the assignment of $\mathbf{x}_i$, $\mathbb{C}$ is initialized as $\mathbb{W}$ (line 5). Algorithm 1 obtains the upper and lower bounds of the inner product for each codeword in $\mathbb{C}$ (line 7), followed by the computation of lower bound $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$ for each codeword (line 8). The algorithm initializes $\mathbf{w}$ as the codeword that minimizes the lower bounding loss and computes its upper bound $\overline{l}(\mathbf{x}_i, \mathbf{w})$ (line 9-10). It utilizes $\overline{l}(\mathbf{x}_i, \mathbf{w})$ to prune other codewords; if $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > \overline{l}(\mathbf{x}_i, \mathbf{w})$, codeword $\mathbf{w}_j$ is pruned from the candidates (lines 11-13). This is because, if $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > \overline{l}(\mathbf{x}_i, \mathbf{w})$, we have $l(\mathbf{x}_i, \mathbf{w}_j) \geq \underline{l}(\mathbf{x}_i, \mathbf{w}_j) > \overline{l}(\mathbf{x}_i, \mathbf{w}) \geq l(\mathbf{x}_i, \mathbf{w})$, and thus codeword $\mathbf{w}_j$ cannot minimize the loss. After pruning, if $\mathbb{C}$ contains only $\mathbf{w}$ (i.e., $\mathbb{C} = \{\mathbf{w}\}$), $\mathbf{w}$ is set as the codeword for $\mathbf{x}_i$ (line 23). Otherwise (i.e., $\mathbb{C} \neq \{\mathbf{w}\}$), the algorithm prunes the codewords by exactly computing $l(\mathbf{x}_i, \mathbf{w})$ (lines 14-22). Specifically, it computes $l(\mathbf{x}_i, \mathbf{w})$ for codeword $\mathbf{w}$ (line 15) and computes the loss of codeword $\mathbf{w}_j$ only if $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) \leq l(\mathbf{x}_i, \mathbf{w})$ (lines 19-20). This is because codeword $\mathbf{w}_j$ cannot minimize the loss if $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > l(\mathbf{x}_i, \mathbf{w})$. In the case of $l(\mathbf{x}_i, \mathbf{w}_j) \leq l(\mathbf{x}_i, \mathbf{w})$, $\mathbf{w}$ is updated to $\mathbf{w}_j$ (line 21-22). The computational complexity of Algorithm 1 is given as follows:

Algorithm 1 Vector Assignment

Input: set of vectors $\mathbb{X}$, set of codewords $\mathbb{W}$, orthonormal matrix $\mathbf{U}$
Output: codeword of each vector

```
1:  for each w_j ∈ 𝕎 do
2:      w'_j = U^⊤ w_j;
3:      ẇ_j = ‖w_j‖^2 − ‖w'_j‖^2;
4:  for each x_i ∈ 𝕏 do
5:      ℂ = 𝕎;
6:      for each w_j ∈ ℂ do
7:          compute p̄(x_i, w_j) and p̲(x_i, w_j) by Equation (13);
8:          compute l̲(x_i, w_j) by Equation (16);
9:      w = argmin_{w_j ∈ ℂ} l̲(x_i, w_j);
10:     compute l̄(x_i, w) by Equation (16);
11:     for each w_j ∈ ℂ\w do
12:         if l̲(x_i, w_j) > l̄(x_i, w) then
13:             subtract w_j from ℂ;
14:     if ℂ ≠ {w} then
15:         compute l(x_i, w) by Equation (5);
16:         for each w_j ∈ ℂ do
17:             if l̲(x_i, w_j) > l(x_i, w) then
18:                 subtract w_j from ℂ;
19:             else
20:                 compute l(x_i, w_j) by Equation (5);
21:                 if l(x_i, w_j) ≤ l(x_i, w) then
22:                     w = w_j;
23:     return w;
```

Lemma 4.6. *If ρ is the ratio of codewords needed in computing exact losses, Algorithm 1 requires $O(\rho ncd + c\log d(n+d))$ time to assign n vectors to codewords that minimize the losses.*

Lemma 4.6 indicates that we can efficiently prune unlikely codewords to assign vectors.

4.2 Efficient Codeword Computation

This section describes our approach to efficiently computing codewords. As shown in Equation (10), the original approach employs $d \times d$ inverse matrix $\mathbf{M}^{-1}$ to compute $\mathbf{w}_j$ directly. This is based on the following equation:

$$\mathbf{M}\mathbf{w}_j = \textstyle\sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp}\mathbf{x}_i. \quad (18)$$

However, this approach has a high computational cost for each codeword, requiring $O(n_j d^2)$ to compute $\mathbf{M}$ and $O(n_j d^2 + d^3)$ to compute $\mathbf{w}_j$ from $\mathbf{M}^{-1}$.

Our approach exploits the conjugate gradient method to avoid the direct use of $\mathbf{M}^{-1}$ in obtaining codewords [2]. This method iteratively updates $\mathbf{w}_j$ using matrix-vector multiplications for $\mathbf{M}$ until convergence, eliminating the need to compute $\mathbf{M}^{-1}$ explicitly. A naive application of the conjugate gradient method for Equation (18) given by matrix $\mathbf{M}$ yields the following equation:

$$\mathbf{P}^{-1}\mathbf{M}\mathbf{w}_j = \mathbf{P}^{-1}\textstyle\sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp}\mathbf{x}_i, \quad (19)$$

where $\mathbf{P}$ is a symmetric and positive definite preconditioning matrix such that $\mathbf{P}^{-1}\mathbf{M} \approx \mathbf{I}$ used to reduce the number of iterations [2].

However, this naive approach has two significant drawbacks. First, it cannot guarantee the exact computation of the codewords. While the conjugate gradient method converges to the exact solution for symmetric positive definite matrices, matrix $\mathbf{M}$ in Equation (19) is not guaranteed to be positive definite. As a result, the solution may not be accurate. Second, computing the preconditioning matrix is costly. A typical choice is the incomplete Cholesky decomposition, which approximates $\mathbf{M}$ as $\mathbf{P} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is a lower triangular matrix. However, the incomplete Cholesky decomposition incurs a time complexity of $O(d^3)$ [9], negating the benefits of using the conjugate gradient method.

To exactly compute codewords, the proposed approach uses the following property of matrix $\mathbf{M}^2$:

Lemma 4.7. *$\mathbf{M}^2$ is a symmetric and positive definite matrix.*

Based on this lemma, we use matrix $\mathbf{M}^2$ instead of $\mathbf{M}$ in applying the conjugate gradient method. Since $\mathbf{M}^2$ is symmetric and positive definite, the conjugate gradient method converges to the exact solution, and thus we can compute codewords exactly. Furthermore, to compute the preconditioning matrix efficiently, we exploit approximated vectors used to assign each vector efficiently in Section 4.1.1. Specifically, let $\mathbf{u}_1$ be the first column vector of matrix $\mathbf{U}$ in Equation (12), we approximate each vector as $\mathbf{x}_i \approx \mathbf{u}_1 x_i'[1]$ and compute $\mathbf{P}$ as follows, based on Equation (11):

$$\mathbf{P} = \left(\mathbf{I}\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp} + \sum_{\mathbf{x}_i\in\mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}\mathbf{u}_1 x_i'[1](\mathbf{u}_1 x_i'[1])^\top\right)^2. \quad (20)$$

Since we have $\mathbf{x}_i \approx \mathbf{u}_1 x_i'[1]$, it is clear that $\mathbf{P}^{-1}\mathbf{M}^2 \approx \mathbf{I}$ from Equations (11) and (20). For matrix $\mathbf{P}$, we have the following property:

Lemma 4.8. *Matrix $\mathbf{P}$ is symmetric and positive definite.*

Since $\mathbf{P}$ is symmetric and positive definite from Lemma 4.8, we can use it as a preconditioning matrix in the conjugate gradient method to compute codewords. Since the size of $\mathbf{P}$ is $d \times d$, it takes $O(d^3)$ time to compute $\mathbf{P}^{-1}$ if we naively compute $\mathbf{P}^{-1}$ from Equation (20) [11]. However, we can efficiently compute $\mathbf{P}^{-1}$ as follows:

Lemma 4.9. *If $\alpha = \sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp}$ and $\beta = \sum_{\mathbf{x}_i\in\mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}(x_i'[1])^2$, $\mathbf{P}^{-1}$ is computed as follows:*

$$\mathbf{P}^{-1} = \frac{1}{\alpha^2}\left(\mathbf{I} + \left(\frac{\beta^2}{(\alpha+\beta)^2} - \frac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right). \quad (21)$$

Note that we need not explicitly compute $\mathbf{P}^{-1}$ using Equation (21) when computing codeword $\mathbf{w}_j$ since it is used in matrix-vector multiplications in the conjugate gradient method.

Algorithm 2 shows our algorithm to compute $\mathbf{w}_j$. In this algorithm, $\mathbf{0}$ is a zero column vector of length d whose elements are 0, and $\mathbf{v}_k$ is a column vector of length d corresponding to the residuals. Length d column vectors $\mathbf{g}$, $\mathbf{y}_k$, and $\mathbf{z}$ are used to obtain the gradient search direction. Algorithm 2 first initializes $\mathbf{v}_0$ from $\mathbf{M}$, $\mathbf{g}$ from matrix $\mathbf{P}^{-1}$, and $\mathbf{y}_0$ from vector $\mathbf{g}$ after computing α and β (line 2-4). It then iteratively updates the column vectors by performing matrix-vector multiplications for $\mathbf{M}$ and $\mathbf{P}^{-1}$ until $\mathbf{v}_k$ converges to a zero column vector (lines 5-12). As shown in Algorithm 2, we do not explicitly compute $\mathbf{M}$, $\mathbf{M}^{-1}$, or $\mathbf{P}^{-1}$, which would incur high computational costs. Instead, we compute matrix-vector multiplications for $\mathbf{M}$ and $\mathbf{P}^{-1}$ to obtain codeword $\mathbf{w}_j$. Algorithm 2 has the following computational cost:

Algorithm 2 Codeword Computation

Input: set of vectors $\mathbb{X}$, weight $h_{i,\parallel}$ and $h_{i,\perp}$, vector $\mathbf{u}_1$
Output: codebook $\mathbb{W}$
1: **for** each $\mathbf{w}_j \in \mathbb{W}$ **do**
2: $\alpha = \sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp}$, $\beta = \sum_{\mathbf{x}_i\in\mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}(x_i'[1])^2$, $k = 0$, $\mathbf{w}_j = \mathbf{0}$;
3: $\mathbf{v}_0 = \left(\mathbf{I}\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp} + \sum_{\mathbf{x}_i\in\mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}\mathbf{x}_i\mathbf{x}_i^\top\right)\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp}\mathbf{x}_i$;
4: $\mathbf{g} = \frac{1}{\alpha^2}\left(\mathbf{I} + \left(\frac{\beta^2}{(\alpha+\beta)^2} - \frac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right)\mathbf{v}_0$, $\mathbf{y}_0 = \mathbf{g}$;
5: **repeat**
6: $k = k + 1$;
7: $\mathbf{z} = \left(\mathbf{I}\sum_{\mathbf{x}_i\in\mathbb{X}_j} h_{i,\perp} + \sum_{\mathbf{x}_i\in\mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}\mathbf{x}_i\mathbf{x}_i^\top\right)^2\mathbf{g}$;
8: $\mathbf{w}_j = \mathbf{w}_j + (\mathbf{y}_{k-1}^\top\mathbf{v}_{k-1}/\mathbf{g}^\top\mathbf{z})\mathbf{g}$;
9: $\mathbf{v}_k = \mathbf{v}_{k-1} - (\mathbf{y}_{k-1}^\top\mathbf{v}_{k-1}/\mathbf{g}^\top\mathbf{z})\mathbf{z}$;
10: $\mathbf{y}_k = \frac{1}{\alpha^2}\left(\mathbf{I} + \left(\frac{\beta^2}{(\alpha+\beta)^2} - \frac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right)\mathbf{v}_k$;
11: $\mathbf{g} = \mathbf{y}_k + (\mathbf{y}_k^\top\mathbf{v}_k/\mathbf{y}_{k-1}^\top\mathbf{v}_{k-1})\mathbf{g}$;
12: **until** $\mathbf{v}_k \approx \mathbf{0}$
13: **return** $\mathbb{W}$;

Lemma 4.10. *Let t_c be the number of iterations of the conjugate gradient method, Algorithm 2 computes codewords in $O(ndt_c)$ time.*

This lemma indicates that we can compute each codeword efficiently without using $\mathbf{M}^{-1}$.

4.3 Vector Quantization

This section presents our algorithm for quantizing vectors.

4.3.1 Quantization Algorithm. Algorithm 3 provides a complete description of our approach. Unlike the original algorithm, which computes the weights $h_{i,\parallel}$ and $h_{i,\perp}$ in $O(d^2)$ time, our approach reduces its computational cost to $O(d)$ from the following lemma:

Lemma 4.11. *Let $I_d = \sin^{d+1}\varphi\cos\varphi$ and $J_d = \int_0^\varphi \sin^d\theta d\theta$, it takes $O(d)$ time to compute parallel weight $h_{i,\parallel}$ and orthogonal weight $h_{i,\perp}$ from the following equation:*

$$h_{i,\parallel} = I_{d-2} + J_d,\ h_{i,\perp} = J_d. \quad (22)$$

where I_d and J_d are incrementally computed as follows:

$$I_d = \begin{cases} I_{d-2}\sin^2\varphi & \text{if } d \geq 2 \\ \sin\varphi\cos\varphi & \text{if } d = 0 \\ \sin^2\varphi\cos\varphi & \text{if } d = 1, \end{cases} \quad J_d = \begin{cases} \frac{d-1}{d}J_{d-2} - \frac{1}{d}I_{d-2} & \text{if } d \geq 2 \\ \varphi & \text{if } d = 0 \\ 1-\cos\varphi & \text{if } d = 1. \end{cases} \quad (23)$$

Using this lemma, our algorithm computes $h_{i,\parallel}$ and $h_{i,\perp}$ for each vector via Equation (22) after initializing the codewords by sampling as shown in Algorithm 3 (line 1-3). It computes the approximated vector from the SVD of randomly sampled vectors (line 4-6). Following the original approach, it iteratively assigns each vector to the codeword that minimizes the loss using Algorithm 1 and computes codewords using Algorithm 2 until convergence (line 7-10). Our approach has the following properties:

Theorem 4.12. *Algorithm 3 takes $O(d(d\log n + n\log d) + t(nd(\rho c + t_c) + c\log d(n+d)))$ time and $O(d(n + c + \log d))$ space to compute c codewords from n vectors.*

Algorithm 3 F-ScaNN

Input: set of vectors $\mathbb{X}$
Output: codebook $\mathbb{W}$
initialize $\mathbb{W}$ by randomly sampling c vectors;
for each $\mathbf{x}_i \in \mathbb{X}$ **do**
  compute $h_{i,\parallel}$ and $h_{i,\perp}$ from Equation (22);
compute rand-log d SVD on S;
for each $\mathbf{x}_i \in \mathbb{X}$ **do**
  $\mathbf{x}'_i = \mathbf{U}^\top \mathbf{x}_i$;
repeat
  compute vector assignment by Algorithm 1;
  compute codewords by Algorithm 2;
until codewords reach the convergence
return $\mathbb{W}$;

Theorem 4.13. *Algorithm 3 theoretically yields the same quantization results as the original approach of ScaNN.*

Theorems 4.12 and 4.13 indicate that Algorithm 3 can efficiently quantize vectors while guaranteeing equivalent quantization results to the original approach and reducing memory cost.

4.3.2 Improving Search Accuracy. As described in the previous section, Algorithm 3 theoretically yields the same quantization results as the original approach. This section describes our approach to improving the search accuracy of the proposed approach.

If $\mathbb{X}_j$ is the set of vectors assigned to codeword $\mathbf{w}_j$, vector $\mathbf{x}_i$ within $\mathbb{X}_j$ is quantized as $\tilde{\mathbf{x}}_i = \mathbf{w}_j$. However, the residual error would be large since all vectors in $\mathbb{X}_j$ are quantized to the same codeword $\mathbf{w}_j$. To address this, we use scaled codewords to reduce the residual error. Specifically, if $\mu_{\mathbf{x}_i}$ is a scaling parameter for the codeword associated with $\mathbf{x}_i$, we quantize $\mathbf{x}_i \in \mathbb{X}_j$ as follows:

$$\mathbf{x}_i \approx \mu_{\mathbf{x}_i} \mathbf{w}_j. \tag{24}$$

We set parameter $\mu_{\mathbf{x}_i}$ to reduce the residual error of $\mathbf{x}_i$. When $\mathbf{x}_i$ is quantized as in Equation (24), the error is given as follows:

$$\begin{aligned}(\mathbf{r}(\mathbf{x}_i, \mu_{\mathbf{x}_i}\mathbf{w}_j))^2 &= (\mathbf{x}_i - \mu_{\mathbf{x}_i}\mathbf{w}_j)^\top (\mathbf{x}_i - \mu_{\mathbf{x}_i}\mathbf{w}_j) \\ &= \|\mathbf{x}_i\|^2 - 2\mu_{\mathbf{x}_i} p(\mathbf{x}_i, \mathbf{w}_j) + \mu_{\mathbf{x}_i}^2 \|\mathbf{w}_j\|^2.\end{aligned} \tag{25}$$

The derivative of the error with respect to $\mu_{\mathbf{x}_i}$ is given by:

$$\frac{\partial (\mathbf{r}(\mathbf{x}_i, \mu_{\mathbf{x}_i}\mathbf{w}_j))^2}{\partial \mu_{\mathbf{x}_i}} = -2p(\mathbf{x}_i, \mathbf{w}_j) + 2\mu_{\mathbf{x}_i}\|\mathbf{w}_j\|^2. \tag{26}$$

The squared residual error is minimized when the derivative is zero, i.e., $\frac{\partial (\mathbf{r}(\mathbf{x}_i, \mu_{\mathbf{x}_i}\mathbf{w}_j))^2}{\partial \mu_{\mathbf{x}_i}} = 0$. Solving this yields:

$$\mu_{\mathbf{x}_i} = \frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{w}_j\|^2}. \tag{27}$$

If we have $p(\mathbf{x}_i, \mathbf{w}_j)$ and $\|\mathbf{w}_j\|^2$, we can obtain $\mu_{\mathbf{x}_i}$ in $O(1)$ time, and memory cost to hold $\mu_{\mathbf{x}_i}$ is $O(1)$, resulting in negligible computational and memory overhead. Note that $\mu_{\mathbf{x}_i}$ is a parameter of FLOAT32 used for each vector. Since we use Equation (24), if $\mathbf{q}$ is quantized to $\tilde{\mathbf{q}}$, we can compute inner product $p(\mathbf{q}, \mathbf{x}_i)$ as follows:

$$p(\mathbf{q}, \mathbf{x}_i) \approx \mu_{\mathbf{q}}\mu_{\mathbf{x}_i} p(\tilde{\mathbf{q}}, \tilde{\mathbf{x}}_i), \tag{28}$$

which can be obtained in $O(1)$ time using a lookup table.

4.3.3 Integration with Pruning Approaches. As described in Section 1, vector quantization can be combined with the pruning approaches to perform MIPS efficiently. This section details how our approach integrates with the pruning approaches.

Since the pruning approaches can efficiently compute candidates, our approach can efficiently verify whether a candidate has the highest inner product. However, since vector quantization approximately computes inner products, directly applying our approach could reduce the search accuracy of the pruning approaches. To address this issue, we compute upper bounds for the inner products from codewords. If the upper bound of a candidate is smaller than the exact inner product of another candidate, we can prune the candidate. By eliminating candidates that cannot have the highest inner product, we maintain the search accuracy of the pruning approaches. If $u(\mathbf{q}, \mathbf{x}_i)$ is the upper bound of $p(\mathbf{q}, \mathbf{x}_i)$ derived from the codewords, the upper bound is computed using the approximated inner product obtained from a lookup table as follows:

$$u(\mathbf{q}, \mathbf{x}_i) = \mu_{\mathbf{q}}\mu_{\mathbf{x}_i} p(\tilde{\mathbf{q}}, \tilde{\mathbf{x}}_i) + \mu_{\mathbf{q}}\|\tilde{\mathbf{q}}\|\|\mathbf{r}_{\mathbf{x}_i}\| + \mu_{\mathbf{x}_i}\|\tilde{\mathbf{x}}_i\|\|\mathbf{r}_{\mathbf{q}}\| + \|\mathbf{r}_{\mathbf{q}}\|\|\mathbf{r}_{\mathbf{x}_i}\|, \tag{29}$$

where $\mathbf{r}_{\mathbf{x}_i} = \mathbf{x}_i - \mu_{\mathbf{x}_i}\tilde{\mathbf{x}}_i$ and $\mathbf{r}_{\mathbf{q}} = \mathbf{q} - \mu_{\mathbf{q}}\tilde{\mathbf{q}}$. We can compute $\|\mathbf{r}_{\mathbf{x}_i}\|$ in $O(d)$ time when quantizing vectors. Similarly, when a query $\mathbf{q}$ is given, $\|\mathbf{r}_{\mathbf{q}}\|$ can be computed in $O(d)$ time. If we have $\|\mathbf{r}_{\mathbf{x}_i}\|$ and $\|\mathbf{r}_{\mathbf{q}}\|$, we can obtain $u(\mathbf{q}, \mathbf{x}_i)$ in $O(1)$ time. The following lemma holds for $u(\mathbf{q}, \mathbf{x}_i)$:

Lemma 4.14. *We have the following equation for $u(\mathbf{q}, \mathbf{x}_i)$:*

$$u(\mathbf{q}, \mathbf{x}_i) \geq p(\mathbf{q}, \mathbf{x}_i). \tag{30}$$

By efficiently and safely pruning unlikely candidates using the upper bounds, we can reduce the processing time of the pruning approaches without sacrificing their search accuracy.

5 Experimental Evaluation

We performed experiments to confirm the effectiveness of our approach. "Proposed" represents the results of our quantization approach. Note that our approach used scaled codewords described in Section 4.3.2. "Original", "PQ", "OPQ", "AAQ", and "QAQ" represent the results of the original approach [6], product quantization [8], optimized product quantization [5], anisotropic additive quantization [16], and query-aware quantization [15], respectively.

The experiments used the eleven datasets: Madelon (MD), MNIST (MN), Fashion-MNIST (FS), Devnagari-Script (DV), aloi (AL), EMNIST_Balanced (EM), kaggle_santander_p (KG), Kuzushiji-49 (KZ), imagenet (IM), tiny5m (TN), and sift10m (SF). These datasets contain 2600, 70000, 70000, 92000, 108000, 131600, 200000, 270912, 2340373, 5000000, and 10000000 vectors with dimensions of 500, 780, 784, 1024, 128, 784, 202, 784, 150, 384, and 128, respectively. We downloaded these datasets from OpenML (https://api.openml.org/) and IRISA (https://www.irisa.fr/). We set the number of codewords to 16, codebooks to 25, and the threshold to 0.2 by following the previous papers [6, 15]. In anisotropic additive quantization, we set the number of codeword updates to three by following [16]. In query-aware quantization, we set the number of held-out samples to 500, the number of vectors sampled by K-means to 2,000, and the number of codeword updates to 20 by following [15]. We implemented all the approaches in C++. We conducted all experiments on a Linux server with Intel Xeon Platinum 8280 CPUs with 2.70 GHz processors. The server had 1.5TB of memory and a 1TB hard disk.

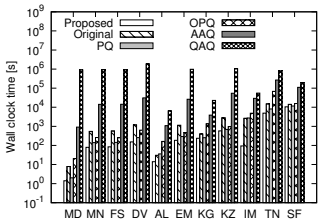
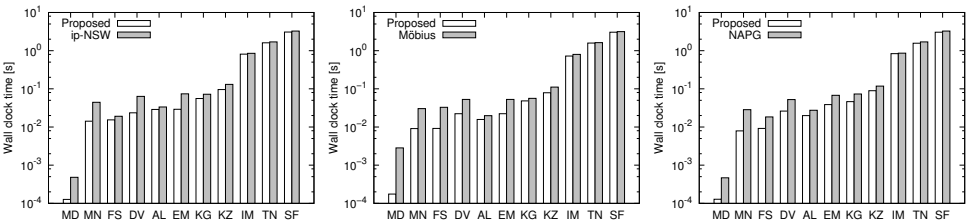


Figure 2: Quantization time of each approach.



(1) ip-NSW (2) Möbius-Graph (3) NAPG

Figure 3: Search time of the graph search approaches.

Table 2: Recall_1@10 of each approach [%].

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	**54.1**	**63.4**	**78.4**	**41.2**	**45.1**	**64.7**	**60.2**	**61.2**	**16.4**	**86.6**	**98.1**
Proposed′	9.30	56.5	15.2	35.5	7.90	31.4	59.7	53.5	6.28	1.34	69.8
Original	9.30	56.5	15.2	35.5	7.90	31.4	59.7	53.5	6.28	1.34	69.8
PQ	9.10	52.1	9.84	32.1	7.67	31.0	56.7	53.3	8.78	26.5	68.6
OPQ	12.1	56.1	27.5	40.1	7.80	48.2	57.8	57.3	10.0	28.0	84.0
AAQ	28.6	7.72	2.61	11.7	4.99	5.31	1.33	10.3	0.84	0.10	73.7
QAQ	0.40	0.70	0.01	11.1	0.03	0.56	0.13	2.28	0.43	0.01	1.11

5.1 Quantization Time

We assessed the quantization time in Figure 2. It shows that our approach is up to 28.1, 28.9, 51.8, 653, and 643000 times faster than the original approach, product quantization, optimized product quantization, anisotropic additive quantization, and query-aware quantization, respectively. As described in Section 3, the original approach iteratively updates codewords from inverse matrices after computing the losses between vectors and codewords. Therefore, it is slower than our approach. Similarly, product quantization and optimized product quantization are slower than our approach. This is because, unlike our approach, they need to compute similarities for all pairs of vectors and codewords. Moreover, since the computational costs to update codewords are high in anisotropic additive quantization and query-aware quantization, they are slower than our approach. On the other hand, we prune unlikely codewords to efficiently assign vectors by using the upper and lower bounds of the losses. Furthermore, we avoid computing the inverse matrices to efficiently compute codewords by exploiting the conjugate gradient method. Figure 2 shows that our approach can effectively reduce quantization time for Madelon dataset, indicating its efficiency for a small dataset. We show the time and space complexity of each approach in Section C.

5.2 Search Accuracy

This experiment examined the search accuracy of each approach. As discussed in Section 4.3.2, we can enhance MIPS search accuracy by incorporating scaled codewords. Since the number of codewords and codebooks was set to 16 and 25, respectively, it needs 100 bits per vector for quantization. In addition, as described in Section 4.3.2, our approach needs an additional scaling parameter represented in FLOAT32. Therefore, if we use the same number of codewords and codebooks, our approach would take 132 bits per vector. To ensure a fair comparison, our approach reduced the number of codebooks to 17, so the total bit usage remains at 100. Following [10], we randomly sampled queries from each dataset, reflecting real-world settings where queries and vectors share similar distributions [15]. Table 2 reports the Recall_1@10 values for each approach. In the table, "Proposed′" refers to our approach without the scaling parameter, using 25 codebooks to match the bit budget of the baseline. We report the Recall_1@100 values for each approach in Section D.

As shown in Table 2, our approach yields the same search accuracy as the original approach if we do not use the scaling parameter. This is because it can obtain the same codewords as the original approach, as shown in Section 4.3.1. The table shows that product quantization is more accurate than ScaNN in imagenet and tiny5m for datasets. This is because their feature values are smaller than other datasets, and ScaNN's loss function ignores vectors with small inner products, leading to higher quantization errors. As described in Section 4.3.2, our approach uses scaled codewords to reduce residual error. As a result, we can more accurately perform MIPS than the original approach. Besides, our approach is more accurate than product quantization and optimized product quantization since it can effectively quantize vectors with higher inner products, the same as ScaNN. In anisotropic additive quantization, since each codebook covers all features without splitting each vector, a large number of vectors have high inner products. Therefore, it fails to reduce quantization losses using the threshold effectively. In addition, query-aware quantization yields lower quantization performance than the proposed approach. This is because, to improve efficiency, query-aware quantization uses only sampled vectors via K-means for quantization instead of all vectors, as mentioned in Section 2.

5.3 Combining with Graph Search Approaches

As discussed in Section 4.3.3, our approach can be combined with existing pruning approaches. This experiment integrated our approach with ip-NSW [10], Möbius-Graph [17], and NAPG [13]. ip-NSW traverses a proximity graph by exploring nodes close to the query and maintaining a search queue. Möbius-Graph is a variant of ip-NSW that applies a Möbius transformation to the vectors prior to graph construction. NAPG, another ip-NSW variant, modifies vector norms during edge selection to construct a proximity graph. Following [13], we set the number of edges per node and the queue length to 16 and 100, respectively. Figure 3 and Table 3 show the search time and Recall_1@10, respectively.

As shown in Figure 3, our approach reduces search time by up to 73.5%, 93.8%, and 72.7% for ip-NSW, Möbius-Graph, and NAPG, respectively. Graph search approaches perform greedy traversal by repeatedly computing exact inner products, leading to high computational cost. Our approach instead computes upper bounds and selectively evaluates exact inner products only for promising candidates. Therefore, it achieves higher efficiency than standalone

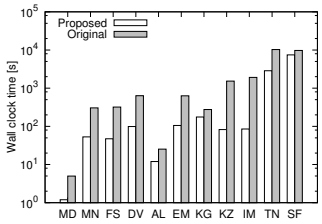


Figure 4: Processing time of the vector assignments.

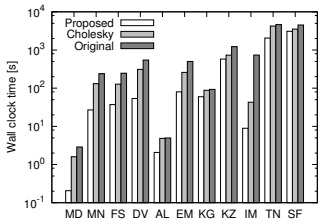


Figure 5: Processing time of the codeword computations.

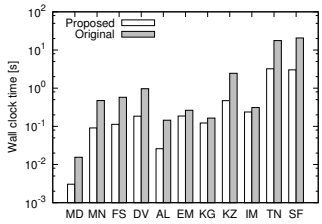


Figure 6: Processing time of the weight computations.

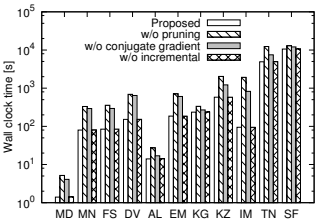


Figure 7: Results of the ablation study.

Table 3: Recall_1@10 of the graph search approaches [%].

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	99.9	85.1	84.8	74.6	7.17	87.5	39.6	85.0	57.6	99.5	99.9
ip-NSW	99.9	85.1	84.8	74.6	7.17	87.5	39.6	85.0	57.6	99.5	99.9

(1) ip-NSW

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	99.5	99.6	74.0	95.3	0.18	98.4	50.2	99.2	91.7	18.1	99.9
Möbius	99.5	99.6	74.0	95.3	0.18	98.4	50.2	99.2	91.7	18.1	99.9

(2) Möbius-Graph

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	99.9	1.58	95.4	77.7	7.11	93.3	39.4	93.5	72.7	99.9	14.4
NAPG	99.9	1.58	95.4	77.7	7.11	93.3	39.4	93.5	72.7	99.9	14.4

(3) NAPG

graph search approaches. As shown in Table 3, it maintains the same search accuracy. This is because, using the upper bounds, we can prune nodes that cannot be traversed in the greedy search.

5.4 Effectiveness of Each Approach

As explained in Section 4.1, vectors can be efficiently assigned by leveraging upper and lower bounds, avoiding exact loss computation for unlikely codewords. In addition, as described in Section 4.2, each codeword is computed using the conjugate gradient method without computing the inverse of matrix $\mathbf{M}$. Furthermore, Section 4.3.1 describes how parallel and orthogonal weights are incrementally computed to enhance efficiency. This section presents experimental results validating the effectiveness of these approaches.

Figure 4 evaluates the processing time of vector assignment during iterations. As shown, our approach reduces processing time by up to 95.5% compared to the original approach, indicating ratio ρ is low. The original approach computes losses for all $n \times c$ vector-codeword pairs, resulting in $O(ncd)$ time complexity. In contrast, we compute lower bounds for all pairs in $O(nc \log d)$ time, as detailed in Algorithm 1. If $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > \bar{l}(\mathbf{x}_i, \mathbf{w})$ holds for all codewords except $\mathbf{w}$, we can determine the codeword of $\mathbf{x}_i$ as $\mathbf{w}$ without exactly computing the losses, as shown in Algorithm 1. Even when this condition is not met, we can prune unlikely codewords by computing the exact loss of codeword $\mathbf{w}$, as shown in Algorithm 1. As a result, we can reduce the number of exact loss evaluations.

Figure 5 shows the processing time required to compute codewords during iterations, with the conjugate gradient tolerance set to 10^{-6}. In this figure, "Cholesky" refers to the incomplete Cholesky decomposition-based preconditioning approach. Our approach achieves up to 98.8% and 87.0% reduction in processing time compared to the original and Cholesky-based approaches, respectively. As detailed in Section 3, computing $\mathbf{M}$ requires $O(n_j d^2)$ time, and its inverse takes $O(d^3)$ time, making the original approach computationally expensive. Although incomplete Cholesky decomposition is used as a preconditioner, it still requires $O(d^3)$ time. In contrast, our approach performs only matrix-vector multiplications with $\mathbf{M}$ and $\mathbf{P}^{-1}$ in $O(n_j d)$ time. Since it avoids explicit computation of $\mathbf{M}$, its inverse matrix, and preconditioning matrix explicitly, we can accelerate codeword computation.

Figure 6 shows the processing time for computing the parallel and orthogonal weights. Our approach reduces the computation time by up to 85.3% relative to the original approach. The original approach takes $O(d^2)$ time to compute the weights for each vector. In contrast, we introduce I_d and J_d that can be computed incrementally, and use them to compute the weights in $O(d)$ time. Therefore, we can efficiently compute the parallel and orthogonal weights.

Figure 7 shows the results of an ablation study under three settings: "w/o pruning", "w/o conjugate gradient", and "w/o incremental" in which we omitted the bounds-based vector assignment, conjugate gradient-based codeword computation, and incremental weight computation, respectively. As shown in Figure 7, among these, "w/o pruning" results in the highest computation time, indicating that bounds-based assignment plays the most significant role in reducing quantization time. This is because the original approach requires computing losses for all vector-codeword pairs to assign vectors, which is the main computational bottleneck.

5.5 Parameter Setting

Our approach requires specifying the number of codewords, codebooks, singular values in the SVD, and threshold T. We evaluate the impact of these parameters using the aloi dataset.

Figures 8 and 9 show the processing time under varying numbers of codewords and codebooks, respectively. As shown in Figure 8, our approach maintains high efficiency even as the number of codewords increases. This is due to its use of upper and lower bounds to efficiently assign vectors and its use of the conjugate gradient method to avoid costly matrix inversion. In contrast, the original approach computes all losses between vectors and codewords. Similarly, product quantization and optimized product quantization compute all similarities between vectors and codewords. Anisotropic additive quantization and query-aware quantization also incur high costs from repeated codeword updates. Since our approach efficiently computes codewords, as shown in Figure 9, it outperforms previous methods in computing codebooks, even when the number of codebooks increases.

Figure 10 presents the processing time of our approach for varying numbers of singular values. As shown in this figure, the processing time is not sensitive to the number of singular values. As

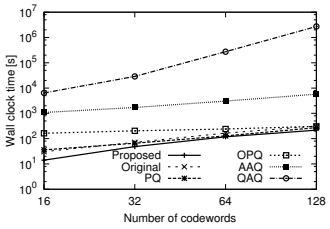


Figure 8: Quantization time vs. the number of codewords.

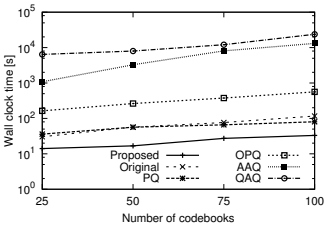


Figure 9: Quantization time vs. the number of codebooks.

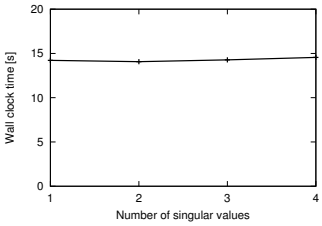


Figure 10: Quantization time vs. the number of singular values.

Table 4: Recall_1@10 for various thresholds [%].

T	Proposed	Original
0.01×mean norm	**49.6**	7.23
0.05×mean norm	**49.3**	7.96
0.1×mean nor m	**48.9**	7.34
0.2×mean norm	**48.4**	7.73
0.3×mean norm	**49.0**	7.39

Table 5: Recall_1@10 for noised data [%].

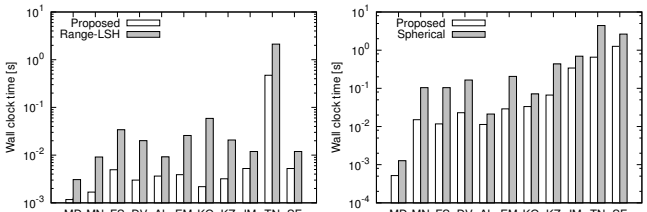
Approach	Standard deviation of Gaussian noise				
	0.1	0.2	0.3	0.4	0.5
Proposed	**93.5**	**100.0**	**100.0**	**100.0**	**100.0**
Original	0.00	0.02	0.03	0.13	0.00
PQ	7.58	11.9	15.3	19.7	28.2
OPQ	4.31	6.51	4.41	3.99	4.11
AAQ	0.00	0.03	0.01	0.01	0.01
QAQ	0.00	0.01	0.00	0.01	0.00

described in Section 4.1, since we use SVD to compute the upper and lower bounds of the losses, we can increase the efficiency of bounds computation if we use a small number of singular values. On the other hand, we can reduce the number of exact loss computations if we use a large number of singular values, since we can improve the accuracy of the bounds. Due to this trade-off involving the number of singular values, our approach is stable for the number of singular values in terms of the processing time.

Table 4 compares the search accuracy of our and original approaches under different threshold settings. As described in the previous studies [15, 16], threshold T was typically chosen from $0.01, 0.05, 0.1, 0.2, 0.3 \times$ the mean vector norm, selecting the value yielding the best performance. This figure shows the search accuracy of each approach for this parameter setting. For fair comparison, our approach adjusted the number of codebooks in to ensure that both used 100 bits. As shown in Table 4, our approach achieves higher search accuracy since we use the scaled codewords to reduce residual error, as described in Section 4.3.2. To further validate the effectiveness of the scaling approach, we evaluated the methods on a noisy version of the aloi dataset by adding Gaussian noise with varying standard deviations in Table 5. This table shows that our approach remains robust under noise, due to the effectiveness of scaled codewords in minimizing residual error.

5.6 Integration with Pruning Approaches

In this experiment, we combined our approach with Norm-Ranging LSH [14] and a spherical k-means-based approach [1]. Norm-Ranging LSH partitions vectors based on their norm distribution and applies hashing to identify candidate vectors. The spherical k-means-based



(1) Range-LSH (2) Spherical

Figure 11: Search time of the pruning approaches.

Table 6: Recall_1@10 of the pruning approaches [%].

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	89.3	58.8	50.8	62.5	55.7	60.3	78.0	56.7	54.8	50.1	54.8
Range-LSH	89.3	58.8	50.8	62.5	55.7	60.3	78.0	56.7	54.8	50.1	54.8

(1) Range-LSH

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	76.9	99.6	75.9	95.3	99.9	97.3	90.6	98.9	98.2	59.9	100
Spherical	76.9	99.6	75.9	95.3	99.9	97.3	90.6	98.9	98.2	59.9	100

(2) Spherical

approach clusters vectors on the unit hypersphere to select candidate vectors. Following [14], we set the number of partitions and hash bits in the LSH approach to 32 and 16, respectively. For the clustering-based approach, we followed [1], setting the number of clusters to $4\sqrt{n}$ and selecting $0.1n$ candidates. Figure 11 and Table 6 present the search time and Recall_1@10, respectively.

As shown in Figure 11, our approach reduces search time by up to 96.3% and 88.8% compared to Norm-Ranging LSH and the spherical k-means-based approach, respectively. While these pruning-based approaches identify candidates, they still require computing exact inner products for all candidates to find the vector with the highest inner product. In contrast, we further improve efficiency by eliminating unlikely candidates using the upper bounds of the inner products. Since vectors that cannot have the maximum inner product are pruned, our approach achieves the same search accuracy as the original pruning approaches, as confirmed in Table 6.

6 Conclusions

In this paper, we addressed the problem of reducing the quantization time of ScaNN. Our approach efficiently assigns vectors by computing the upper and lower bounds of the losses. Furthermore, we compute each codeword by exploiting the conjugate gradient method. Experiments showed that our approach is faster than the previous approaches, yielding accurate quantization results.

References

[1] Sebastian Bruch, Franco Maria Nardini, Amir Ingber, and Edo Liberty. 2024. Bridging Dense and Sparse Maximum Inner Product Search. *ACM Trans. Inf. Syst.* 42, 6 (2024), 151:1–151:38.

[2] James W. Demmel. 2017. *Applied Numerical Linear Algebra.* Orient Blackswan.

[3] Yasuhiro Fujiwara, Yasutoshi Ida, Junya Arai, Mai Nishimura, and Sotetsu Iwamura. 2016. Fast Algorithm for the Lasso based L1-Graph Construction. *Proc. VLDB Endow.* 10, 3 (2016), 229–240.

[4] Yasuhiro Fujiwara, Go Irie, Shari Kuroyama, and Makoto Onizuka. 2014. Scaling Manifold Ranking Based Image Retrieval. *Proc. VLDB Endow.* 8, 4 (2014), 341–352.

[5] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR.* 2946–2953.

[6] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *ICML.* 3887–3896.

[7] Yasutoshi Ida, Sekitoshi Kanai, Yasuhiro Fujiwara, Tomoharu Iwata, Koh Takeuchi, and Hisashi Kashima. 2020. Fast Deterministic CUR Matrix Decomposition with Accuracy Assurance. In *ICML.* 4594–4603.

[8] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.

[9] Stephen Wright Jorge Nocedal. 2006. *Numerical Optimization.* Springer.

[10] Stanislav Morozov and Artem Babenko. 2018. Non-metric Similarity Graphs for Maximum Inner Product Search. In *NeurIPS.* 4726–4735.

[11] William H. Press, Saul A. Teukolsky, William T Vetterling, and Brian P. Flannery. 2007. *Numerical Recipes 3rd Edition: The Art of Scientific Computing.* Cambridge University Press.

[12] Mohammadsaleh Refahi, Bahrad A Sokhansanj, Joshua C Mell, James R Brown, Hyunwoo Yoo, Gavin Hearne, and Gail L Rosen. 2025. Enhancing Nucleotide Sequence Representations in Genomic Analysis with Contrastive Optimization. *Communications Biology* 8, 1 (2025), 517.

[13] Shulong Tan, Zhaozhuo Xu, Weijie Zhao, Hongliang Fei, Zhixin Zhou, and Ping Li. 2021. Norm Adjusted Proximity Graph for Fast Inner Product Retrieval. In *KDD.* ACM, 1552–1560.

[14] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *NeurIPS.* 2956–2965.

[15] Jin Zhang, Defu Lian, Haodi Zhang, Baoyun Wang, and Enhong Chen. 2023. Query-Aware Quantization for Maximum Inner Product Search. In *AAAI.* 4875–4883.

[16] Jin Zhang, Qi Liu, Defu Lian, Zheng Liu, Le Wu, and Enhong Chen. 2022. Anisotropic Additive Quantization for Fast Inner Product Search. In *AAAI.* 4354–4362.

[17] Zhixin Zhou, Shulong Tan, Zhaozhuo Xu, and Ping Li. 2019. Möbius Transformation for Fast Inner Product Search on Graph. In *NeurIPS.* 8216–8227.

A Ethical Use of Data

This paper adheres to all ACM Publications Policies. All experiments were conducted using publicly available and anonymized datasets.

B Proofs

We provide proofs for the lemmas and theorems.

Proof of Lemma 4.2. Since SVD is an orthonormal transformation, we have the following equation for $\mathbf{x}_i$, $\mathbf{w}_j$, $\mathbf{x}'_i$, and $\mathbf{w}'_j$:

$$\|\mathbf{x}_i + \mathbf{w}_j\|^2 = \sum_{k=1}^{d}(x_i[k] + w_j[k])^2 = \sum_{k=1}^{d}(x'_i[k] + w'_j[k])^2.$$

We have the following equation for $\sum_{k=1}^{d}(x_i[k] + w_j[k])^2$:

$$\sum_{k=1}^{d}(x_i[k] + w_j[k])^2 = \|\mathbf{x}_i\|^2 + \|\mathbf{w}_j\|^2 + 2p(\mathbf{x}_i, \mathbf{w}_j).$$

Furthermore, the following equation holds for $\sum_{k=1}^{d}(x'_i[k]+w'_j[k])^2$:

$$\sum_{k=1}^{d}(x'_i[k]+w'_j[k])^2 = \sum_{k=1}^{\log d}(x'_i[k]+w'_j[k])^2 + \sum_{k=\log d+1}^{d}(x'_i[k]+w'_j[k])^2$$
$$= \|\mathbf{x}'_i\|^2 + \|\mathbf{w}'_j\|^2 + 2p(\mathbf{x}'_i, \mathbf{w}'_j) + \sum_{k=\log d+1}^{d}((x'_i[k])^2 + (w'_j[k])^2 + 2x'_i[k]w'_j[k]).$$

Therefore, we have

$$p(\mathbf{x}_i, \mathbf{w}_j) = p(\mathbf{x}'_i, \mathbf{w}'_j) - \tfrac{1}{2}(\|\mathbf{x}_i\|^2 - \|\mathbf{x}'_i\|^2) - \tfrac{1}{2}(\|\mathbf{w}_j\|^2 - \|\mathbf{w}'_j\|^2) + \tfrac{1}{2}\sum_{k=\log d+1}^{d}((x'_i[k])^2 + (w'_j[k])^2 + 2x'_i[k]w'_j[k]).$$

From the Cauchy-Schwarz inequality, we have

$$\sum_{k=\log d+1}^{d} x'_i[k]w'_j[k] \le \sqrt{\sum_{k=\log d+1}^{d}(x'_i[k])^2}\sqrt{\sum_{k=\log d+1}^{d}(w'_j[k])^2}.$$

As a result, we have

$$\sum_{k=\log d+1}^{d}((x'_i[k])^2 + (w'_j[k])^2 + 2x'_i[k]w'_j[k]) \le \left(\sqrt{\sum_{k=\log d+1}^{d}(x'_i[k])^2} + \sqrt{\sum_{k=\log d+1}^{d}(w'_j[k])^2}\right)^2.$$

Additionally, since SVD is an orthonormal transformation, we have

$$\sum_{k=\log d+1}^{d}(x'_i[k])^2 = \sum_{k=1}^{d}(x_i[k])^2 - \sum_{k=1}^{\log d}(x'_i[k])^2 = \dot{x}_i,$$
$$\sum_{k=\log d+1}^{d}(w'_j[k])^2 = \sum_{k=1}^{d}(w_j[k])^2 - \sum_{k=1}^{\log d}(w'_j[k])^2 = \dot{w}_j.$$

As a result, since $\dot{x}_i = \|\mathbf{x}_i\|^2 - \|\mathbf{x}'_i\|^2$ and $\dot{w}_j = \|\mathbf{w}_j\|^2 - \|\mathbf{w}'_j\|^2$,

$$p(\mathbf{x}_i, \mathbf{w}_j) \le p(\mathbf{x}'_i, \mathbf{w}'_j) - \tfrac{1}{2}\dot{x}_i - \tfrac{1}{2}\dot{w}_j + \tfrac{1}{2}(\sqrt{\dot{x}_i} + \sqrt{\dot{w}_j}) = \overline{p}(\mathbf{x}_i, \mathbf{w}_j).$$

Similarly, the following equation holds:

$$\sum_{k=\log d+1}^{d} x'_i[k]w'_j[k] \ge -\sqrt{\sum_{k=\log d+1}^{d}(x'_i[k])^2}\sqrt{\sum_{k=\log d+1}^{d}(w'_j[k])^2}.$$

As a result, we have the following equation:

$$p(\mathbf{x}_i, \mathbf{w}_j) \ge p(\mathbf{x}'_i, \mathbf{w}'_j) - \tfrac{1}{2}\dot{x}_i - \tfrac{1}{2}\dot{w}_j + \tfrac{1}{2}(\sqrt{\dot{x}_i} - \sqrt{\dot{w}_j}) = \underline{p}(\mathbf{x}_i, \mathbf{w}_j),$$

which completes the proof. □

Proof of Lemma 4.3. Since d-dimensional codeword $\mathbf{w}_j$ is approximated as $\mathbf{w}'_j = \mathbf{U}^\top \mathbf{w}_j$ where $\mathbf{U}$ is a matrix of size $d \times \log d$, it takes $O(cd \log d)$ time to obtain c approximated codewords. It takes $O(cd)$ to compute $\dot{w}_j$ for c codewords since $\dot{w}_j = \|\mathbf{w}_j\|^2 - \|\mathbf{w}'_j\|^2$. Additionally, we can compute $\mathbf{x}'_i$ and $\dot{x}_i$ before entering the iterations. Since the computational cost of $p(\mathbf{x}'_i, \mathbf{w}'_j)$ is $O(\log d)$ from Equation (13), it requires $O(nc \log d)$ time to compute the upper and lower bounds for the pairs of n vectors and c codewords from Equation (13). As a result, we can compute the upper and lower bounds in $O(c(n + d) \log d)$ time in each iteration. □

Proof of Lemma 4.5. Since $\mathbf{r}(\mathbf{x}_i, \mathbf{w}_j) = \mathbf{x}_i - \mathbf{w}_j$, we derive the following equation from Equation (3) for the parallel residual error:

$$\mathbf{r}_\parallel(\mathbf{x}_i, \mathbf{w}_j) = \frac{p(\mathbf{x}_i, \mathbf{x}_i - \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\mathbf{x}_i = \frac{\|\mathbf{x}_i\|^2 - p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\mathbf{x}_i = \left(1 - \frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\right)\mathbf{x}_i.$$

Therefore, we have

$$\|\mathbf{r}_\parallel(\mathbf{x}_i, \mathbf{w}_j)\|^2 = \left(1 - \frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\right)^2\|\mathbf{x}_i\|^2 = \|\mathbf{x}_i\|^2 - 2p(\mathbf{x}_i, \mathbf{w}_j) + \frac{(p(\mathbf{x}_i, \mathbf{w}_j))^2}{\|\mathbf{x}_i\|^2}.$$

The following equation holds for the orthogonal residual error:

$$\mathbf{r}_\perp(\mathbf{x}_i, \mathbf{w}_j) = \mathbf{r}(\mathbf{x}_i, \mathbf{w}_j) - \mathbf{r}_\parallel(\mathbf{x}_i, \mathbf{w}_j) = \frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\mathbf{x}_i - \mathbf{w}_j.$$

Therefore, we have

$$\|\mathbf{r}_\perp(\mathbf{x}_i, \mathbf{w}_j)\|^2 = \left(\frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\mathbf{x}_i^\top - \mathbf{w}_j^\top\right)\left(\frac{p(\mathbf{x}_i, \mathbf{w}_j)}{\|\mathbf{x}_i\|^2}\mathbf{x}_i - \mathbf{w}_j\right) = \|\mathbf{w}_j\|^2 - \frac{(p(\mathbf{x}_i, \mathbf{w}_j))^2}{\|\mathbf{x}_i\|^2}.$$

As a result, the score-aware quantization loss is given as follows:

$$l(\mathbf{x}_i, \mathbf{w}_j) = h_{i,\parallel}\|\mathbf{r}_\parallel(\mathbf{x}_i, \mathbf{w}_j)\|^2 + h_{i,\perp}\|\mathbf{r}_\perp(\mathbf{x}_i, \mathbf{w}_j)\|^2$$
$$= \frac{h_{i,\parallel} - h_{i,\perp}}{\|\mathbf{x}_i\|^2}(p(\mathbf{x}_i, \mathbf{w}_j))^2 - 2h_{i,\parallel}p(\mathbf{x}_i, \mathbf{w}_j) + h_{i,\parallel}\|\mathbf{x}_i\|^2 + h_{i,\perp}\|\mathbf{w}_j\|^2.$$

Therefore, if $\lambda = p(\mathbf{x}_i, \mathbf{w}_j)$, we have $L_{i,j}(\lambda) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) = l(\mathbf{x}_i, \mathbf{w}_j)$. Since $h_{i,\parallel} - h_{i,\perp} > 0$ as shown in [6], we have $\frac{h_{i,\parallel} - h_{i,\perp}}{\|\mathbf{x}_i\|^2} > 0$. Therefore, $L_{i,j}(\lambda)$ of Equation (15) is a downward convex quadratic

function for λ. In addition, since $\frac{\partial L_{i,j}(\lambda)}{\partial \lambda} = 2\frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}\left(\lambda - \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}\right)$, we have $\frac{\partial L_{i,j}(\lambda)}{\partial \lambda} = 0$ when $\lambda = \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$.

We have $\overline{p}(\mathbf{x}_i, \mathbf{w}_j) \geq \underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ from Lemma 4.2, and $\frac{1}{2}(\overline{p}(\mathbf{x}_i, \mathbf{w}_j) + \underline{p}(\mathbf{x}_i, \mathbf{w}_j))$ is an intermediate value between $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$. Therefore, if $\frac{1}{2}(\overline{p}(\mathbf{x}_i, \mathbf{w}_j) + \underline{p}(\mathbf{x}_i, \mathbf{w}_j)) \geq \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$, we have

$$\left|\overline{p}(\mathbf{x}_i, \mathbf{w}_j) - \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}\right| \geq \left|p(\mathbf{x}_i, \mathbf{w}_j) - \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}\right|.$$

This equation indicates that $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ is not closer to $\frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$ than $p(\mathbf{x}_i, \mathbf{w}_j)$. As a result, if $\frac{1}{2}(\overline{p}(\mathbf{x}_i, \mathbf{w}_j) + \underline{p}(\mathbf{x}_i, \mathbf{w}_j)) \geq \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$,

$$l(\mathbf{x}_i, \mathbf{w}_j) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) \leq L_{i,j}(\overline{p}(\mathbf{x}_i, \mathbf{w}_j)).$$

Otherwise, since $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ is farther than $p(\mathbf{x}_i, \mathbf{w}_j)$ from $\frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$, the following equation holds:

$$l(\mathbf{x}_i, \mathbf{w}_j) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) \leq L_{i,j}(\underline{p}(\mathbf{x}_i, \mathbf{w}_j)).$$

In addition, if $\overline{p}(\mathbf{x}_i, \mathbf{w}_j) < \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$ holds, we have

$$p(\mathbf{x}_i, \mathbf{w}_j) \leq \overline{p}(\mathbf{x}_i, \mathbf{w}_j) < \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}.$$

As a result, we have the following equation:

$$l(\mathbf{x}_i, \mathbf{w}_j) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) \geq L_{i,j}(\overline{p}(\mathbf{x}_i, \mathbf{w}_j)).$$

Similarly, if we have $\underline{p}(\mathbf{x}_i, \mathbf{w}_j) > \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$, and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ is closer to $\frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$ than $p(\mathbf{x}_i, \mathbf{w}_j)$, we have

$$l(\mathbf{x}_i, \mathbf{w}_j) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) \geq L_{i,j}(\underline{p}(\mathbf{x}_i, \mathbf{w}_j)).$$

Moreover, if $\underline{p}(\mathbf{x}_i, \mathbf{w}_j) \leq \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}} \leq \overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ holds, since we have $\frac{\partial L_{i,j}(\lambda)}{\partial \lambda} = 0$ when $\lambda = \frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}$, the following equation holds:

$$l(\mathbf{x}_i, \mathbf{w}_j) = L_{i,j}(p(\mathbf{x}_i, \mathbf{w}_j)) \geq L_{i,j}\left(\frac{h_{i,\parallel}\|\mathbf{x}_i\|^2}{h_{i,\parallel}-h_{i,\perp}}\right),$$

which completes the proof. □

Proof of Lemma 4.6. Algorithm 1 first computes $\mathbf{w}'_j$ and $\dot{w}_j$ for each codeword and then computes $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ for each pair of n vectors and c codewords. The computational cost of these processes is $O(c \log d(n+d))$ as detailed in Lemma 4.3. As described in Section 4.1.2, once $\overline{p}(\mathbf{x}_i, \mathbf{w}_j)$ and $\underline{p}(\mathbf{x}_i, \mathbf{w}_j)$ are computed, $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$ can be computed in $O(1)$ time. Therefore, it needs $O(nc)$ time to compute $\underline{l}(\mathbf{x}_i, \mathbf{w}_j)$ for each pair. Similarly, it takes $O(n)$ time to compute $\overline{l}(\mathbf{x}_i, \mathbf{w})$ for codeword $\mathbf{w}$. Moreover, it takes $O(\rho ncd)$ time to compute $l(\mathbf{x}_i, \mathbf{w}_j)$ after pruning codewords using the upper and lower bounds of the losses. Thus, the total computational cost of Algorithm 1 is $O(\rho ncd + c \log d(n+d))$. □

Proof of Lemma 4.7. From Equation (11), we have

$$\mathbf{M}^\top = \mathbf{I}^\top \sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp} + \sum_{\mathbf{x}_i \in \mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}(\mathbf{x}_i\mathbf{x}_i^\top)^\top = \mathbf{M}.$$

This equation indicates that $\mathbf{M}$ is a symmetric matrix. Therefore, it is clear that $\mathbf{M}^2$ is a symmetric matrix. Furthermore, since $\mathbf{M}$ has an inverse matrix, none of its eigenvalues are zero [2]. As a result, if σ_i ($\neq 0$) is the i-th eigenvalue of $\mathbf{M}$ and $\mathbf{Q}$ is a $d \times d$ matrix of eigenvectors of $\mathbf{M}$, we have $\mathbf{M} = \mathbf{Q}\,\mathrm{diag}\,(\sigma_1, \ldots, \sigma_d)\mathbf{Q}^\top$, where $\mathrm{diag}\,(\cdot)$ represents a diagonal matrix. Consequently, we have

$$\mathbf{M}^2 = \mathbf{Q}\,\mathrm{diag}(\sigma_1, \ldots, \sigma_d)\mathbf{Q}^\top\mathbf{Q}\,\mathrm{diag}(\sigma_1, \ldots, \sigma_d)\mathbf{Q}^\top = \mathbf{Q}\,\mathrm{diag}(\sigma_1^2, \ldots, \sigma_d^2)\mathbf{Q}^\top.$$

Since $\sigma_i^2 > 0$, this equation indicates that $\mathbf{M}^2$ is positive definite. □

Proof of Lemma 4.9. From Equation (20), we have

$$\mathbf{P} = \left(\mathbf{I}\sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp} + \mathbf{u}_1\left(\sum_{\mathbf{x}_i \in \mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}(x'_i[1])^2\right)\mathbf{u}_1^\top\right)^2 = (\alpha\mathbf{I} + \mathbf{u}_1\beta\mathbf{u}_1^\top)^2.$$

Since $\mathbf{u}_1^\top\mathbf{u}_1 = 1$, we have the following equation from the Sherman-Morrison formula [11]:

$$\mathbf{P}^{-1} = \left(\tfrac{1}{\alpha}\mathbf{I} - \tfrac{1}{\alpha}\mathbf{I}\mathbf{u}_1\beta\left(\beta + \beta\mathbf{u}_1^\top\tfrac{1}{\alpha}\mathbf{I}\mathbf{u}_1\beta\right)^{-1}\beta\mathbf{u}_1^\top\tfrac{1}{\alpha}\mathbf{I}\right)^2 = \tfrac{1}{\alpha^2}\left(\mathbf{I} - \tfrac{\beta}{\alpha+\beta}\mathbf{u}_1\mathbf{u}_1^\top\right)^2$$
$$= \tfrac{1}{\alpha^2}\left(\mathbf{I} - \tfrac{2\beta}{\alpha+\beta}\mathbf{u}_1\mathbf{u}_1^\top + \tfrac{\beta^2}{(\alpha+\beta)^2}\mathbf{u}_1\mathbf{u}_1^\top\mathbf{u}_1\mathbf{u}_1^\top\right) = \tfrac{1}{\alpha^2}\left(\mathbf{I} + \left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right),$$

which completes the proof. □

Proof of Lemma 4.8. If $\mathbf{P}^{-1}$ is a symmetric and positive definite matrix, $\mathbf{P}$ is also a symmetric and positive definite matrix. Therefore, we show that matrix $\mathbf{P}^{-1}$ is symmetric and positive definite. We have the following equation for $\mathbf{P}^{-1}$:

$$(\mathbf{P}^{-1})^\top = \tfrac{1}{\alpha^2}\left(\mathbf{I} + \left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right)^\top$$
$$= \tfrac{1}{\alpha^2}\left(\mathbf{I}^\top + \left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)(\mathbf{u}_1\mathbf{u}_1^\top)^\top\right) = \tfrac{1}{\alpha^2}\left(\mathbf{I} + \left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)\mathbf{u}_1\mathbf{u}_1^\top\right) = \mathbf{P}^{-1}.$$

As a result, $\mathbf{P}^{-1}$ is a symmetric matrix. Furthermore, if $\mathbf{U}_d$ is a $d \times d$ unitary matrix having d left singular vectors of $\mathbf{S}$ and $\mathbf{b}$ is a column vector of length d such that $\mathbf{b} = [1, 0, \ldots, 0]^\top$, we have $\mathbf{U}_d\mathbf{U}_d^\top = \mathbf{I}$ and $\mathbf{u}_1 = \mathbf{U}_d\mathbf{b}$. Therefore, we have

$$\mathbf{P}^{-1} = \left(\tfrac{1}{\alpha^2}\mathbf{U}_d\mathbf{U}_d^\top + \tfrac{1}{\alpha^2}\left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)\mathbf{U}_d\mathbf{b}\mathbf{b}^\top\mathbf{U}_d^\top\right)$$
$$= \mathbf{U}_d\left(\tfrac{1}{\alpha^2}\mathbf{I} + \tfrac{1}{\alpha^2}\left(\tfrac{\beta^2}{(\alpha+\beta)^2} - \tfrac{2\beta}{\alpha+\beta}\right)\mathbf{b}\mathbf{b}^\top\right)\mathbf{U}_d^\top.$$

In this equation, $\mathbf{b}\mathbf{b}^\top$ is a $d \times d$ matrix where the $(1,1)$ element is 1 and the other elements are 0 since $\mathbf{b} = [1, 0, \ldots, 0]^\top$. Thus, we have

$$\mathbf{P}^{-1} = \mathbf{U}_d\,\mathrm{diag}\left(\tfrac{1}{\alpha^2}\left(1 - \tfrac{2\beta}{\alpha+\beta} + \tfrac{\beta^2}{(\alpha+\beta)^2}\right), \tfrac{1}{\alpha^2}, \ldots, \tfrac{1}{\alpha^2}\right)\mathbf{U}_d^\top$$
$$= \mathbf{U}_d\,\mathrm{diag}\left(\tfrac{1}{(\alpha+\beta)^2}, \tfrac{1}{\alpha^2}, \ldots, \tfrac{1}{\alpha^2}\right)\mathbf{U}_d^\top.$$

Since we have $h_{i,\perp} > 0$ [6], we have $\alpha = \sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp} > 0$. Besides, since $h_{i,\parallel} > h_{i,\perp}$ holds [6], we have $\beta = \sum_{\mathbf{x}_i \in \mathbb{X}_j} \frac{h_{i,\parallel}-h_{i,\perp}}{\|\mathbf{x}_i\|^2}(x'_i[1])^2 \geq 0$. Therefore, since $\frac{1}{(\alpha+\beta)^2}, \frac{1}{\alpha^2} > 0$, matrix $\mathbf{P}^{-1}$ is positive definite. □

Proof of Lemma 4.10. Since n_j vectors are included in $\mathbb{X}_j$, it needs $O(n_j)$ time to compute α and β. Since it requires $O(n_j d)$ time to compute $\sum_{\mathbf{x}_i \in \mathbb{X}_j} h_{i,\perp}\mathbf{x}_i$, Algorithm 2 initializes $\mathbf{v}_0$ in $O(n_j d)$ time. Additionally, it initializes $\mathbf{g}$ and $\mathbf{y}_0$ in $O(d)$ time. In each iteration, it computes $\mathbf{z}$ in $O(n_j d)$ time and computes $\mathbf{w}_j$, $\mathbf{v}_k$, $\mathbf{y}_k$, and $\mathbf{g}$ in $O(d)$ time. As a result, since $\sum_{j=1}^{c} n_j = n$, the computational cost of Algorithm 2 is $O(ndt_c)$. □

Proof of Lemma 4.11. Since $I_d = \sin^{d+1}\varphi\cos\varphi$, we have

$$I_d = \sin^{d+1}\varphi\cos\varphi = \sin^{d-1}\varphi\cos\varphi\sin^2\varphi = I_{d-2}\varphi\sin^2\varphi.$$

Additionally, since $I_d = \sin^{d+1}\varphi\cos\varphi$, we have

$$I_0 = \sin\varphi\cos\varphi,\ \ I_1 = \sin^2\varphi\cos\varphi.$$

Therefore, I_d can be computed by Equation (23). Furthermore, we have the following equation from partial integration since $\int \sin^d \theta d\theta = \int \sin^{d-1}\theta \sin\theta d\theta$:

$$\int \sin^d \theta d\theta = (d-1)\left(\int \sin^{d-2} d\theta - \int \sin^d d\theta\right) - \sin^{d-1}\theta\cos\theta.$$

Therefore, we have

$$\int \sin^d \theta d\theta = \tfrac{d-1}{d}\int \sin^{d-2}\theta d\theta - \tfrac{1}{d}\sin^{d-1}\theta\cos\theta.$$

Since $I_d = \sin^{d+1}\varphi\cos\varphi$ and $J_d = \int_0^\varphi \sin^d \theta d\theta$, we have

$$\begin{aligned} J_d &= \tfrac{d-1}{d}\int_0^\varphi \sin^{d-2}\theta d\theta - \left[\tfrac{1}{d}\sin^{d-1}\theta\cos\theta\right]_0^\varphi \\ &= \tfrac{d-1}{d}J_{d-2} - \tfrac{1}{d}\sin^{d-1}\varphi\cos\varphi = \tfrac{d-1}{d}J_{d-2} - \tfrac{1}{d}I_{d-2}. \end{aligned}$$

Moreover, the following equation holds since $J_d = \int_0^\varphi \sin^d \theta d\theta$:

$$J_0 = \varphi,\ J_1 = 1 - \cos\varphi.$$

Consequently, we can compute J_d from Equation (23). In addition, we have the following equation:

$$J_{d-2} = \tfrac{d}{d-1}J_d + \tfrac{1}{d-1}I_{d-2}.$$

Since $J_d = \int_0^\varphi \sin^d \theta d\theta$, the following equation holds from Equation (6) for parallel weight $h_{i,\parallel}$:

$$\begin{aligned} h_{i,\parallel} &= (d-1)\int_0^\varphi (\sin^{d-2}\theta - \sin^d\theta)d\theta = (d-1)(J_{d-2} - J_d) \\ &= dJ_d + I_{d-2} - (d-1)J_d = I_{d-2} + J_d. \end{aligned}$$

Similarly, we have the following equation from Equation (6) for orthogonal weight $h_{i,\perp}$:

$$h_{i,\perp} = \int_0^\varphi \sin^d \theta d\theta = J_d.$$

As shown in Equation (23), we can initialize J_d in $O(1)$ time, and we can incrementally compute J_d from J_{d-2} in $O(1)$ time. Therefore, we can compute J_d in $O(d)$ time. Similarly, we can compute I_d in $O(d)$ time from Equation (23). As a result, parallel weight $h_{i,\parallel}$ and orthogonal weight $h_{i,\perp}$ of vector $\mathbf{x}_i$ can be computed in $O(d)$ time from Equation (23) and (22). □

Proof of Theorem 4.12. It needs $O(c)$ time to sample c vectors to initialize codewords. As shown in Lemma 4.11, we can compute the parallel and orthogonal weights of vector $\mathbf{x}_i$ in $O(d)$ time. Therefore, we can compute the weights of n vectors in $O(nd)$ time. Since the size of matrix $\mathbf{S}$ is $d \times \log n$, we can compute SVD on matrix $\mathbf{S}$ in $O(d^2 \log n)$ time. Additionally, since the size of matrix $\mathbf{U}$ is $d \times \log d$, it needs $O(nd \log d)$ time to approximate n vectors. As shown in Lemma 4.6, the computational cost of data assignment is $O(\rho ncd + c\log d(n+d))$. Furthermore, we can compute codewords in $O(ndt_c)$ time as shown in Lemma 4.10. Consequently, it requires $O(d(d\log n + n\log d) + t(nd(\rho c + t_c) + c\log d(n+d)))$ time.

As for memory cost, it takes $O(nd)$, $O(cd)$, and $O(n)$ space to hold vectors, codewords, and the parallel and orthogonal weights, respectively. As shown in Equation (22), the parallel and orthogonal weights are obtained from variable I_d and J_d. Since these variables are incrementally computed from Equation (23), it needs $O(1)$ space to hold I_d and J_d. As shown in Algorithm 1, we need to additionally hold $\mathbf{U}$, $\mathbf{w}'_j$, and $\dot{w}_j$ when assigning vectors, and their memory costs are $O(d\log d)$, $O(c\log d)$, $O(c)$, respectively. In Algorithm 2, it additionally takes $O(d)$ space to hold vectors $\mathbf{v}_k$, $\mathbf{g}$, $\mathbf{y}_k$, and $\mathbf{z}$. As a result, the memory cost of Algorithm 3 is $O(d(n + c + \log d))$. □

Table 7: Complexity of each approach.

Approach	
Proposed	$O(d(d\log n + n\log d) + t(nd(\rho c + t_c) + c\log d(n+d)))$
Original	$O(dt(nc + nd + cd^2))$
PQ	$O(ncdt)$
OPQ	$O(t(nd(n+c) + d^3))$
AAQ	$O(t(nmd^2 + cmd^3))$
QAQ	$O(t(nd^2 + c^3d^3))$

(1) Time complexity

Approach	
Proposed	$O(d(n + c + \log d))$
Original	$O(d(n + c + d))$
PQ	$O(d(n + c))$
OPQ	$O(d(n + c + d))$
AAQ	$O(d(n + cm + d)$
QAQ	$O(d(n + c^2d) + cm^2)$

(2) Space complexity

Table 8: Recall_1@100 of each approach [%].

Approach	MD	MN	FS	DV	AL	EM	KG	KZ	IM	TN	SF
Proposed	**62.4**	**94.7**	**93.1**	**85.3**	**99.0**	**94.1**	**87.8**	**90.5**	**36.2**	**97.9**	**99.3**
Original	11.9	88.6	38.1	75.5	74.8	75.3	84.4	87.3	7.29	3.22	91.5
PQ	15.0	91.5	37.4	73.9	75.4	77.6	85.0	89.5	25.7	43.5	90.9
OPQ	18.2	91.7	48.6	84.2	75.7	85.0	86.5	90.2	27.6	43.8	91.4
AAQ	34.0	19.8	10.0	22.0	14.6	7.09	3.78	12.3	2.84	0.22	94.8
QAQ	16.1	2.42	0.01	16.2	0.05	1.89	0.46	4.46	1.39	0.02	4.68

Proof of Theorem 4.13. Since Equation (22), used to compute the parallel and orthogonal weights, is derived from the definite integrals in Equation (6), Equation (22) yields theoretically accurate weights. As shown in Algorithm 1, we prune codeword $\mathbf{w}_j$ when assigning vector $\mathbf{x}_i$ if we have $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > \bar{l}(\mathbf{x}_i, \mathbf{w})$ or $\underline{l}(\mathbf{x}_i, \mathbf{w}_j) > l(\mathbf{x}_i, \mathbf{w})$. If this condition holds, we have $l(\mathbf{x}_i, \mathbf{w}_j) > l(\mathbf{x}_i, \mathbf{w})$ from Lemma 4.2. Therefore, the pruned codewords cannot be the codeword that minimizes the loss for vector $\mathbf{x}_i$. As a result, Algorithm 1 exactly assigns vectors. Additionally, our approach can exactly compute codewords using Algorithm 2 since the conjugate gradient method theoretically converges to the exact solution [2]. Consequently, Algorithm 3 theoretically outputs the same quantization results as the original approach of ScaNN. □

Proof of Lemma 4.14. Since $\mathbf{r}_\mathbf{q} = \mathbf{q} - \mu_\mathbf{q}\tilde{\mathbf{q}}$ and $\mathbf{r}_{\mathbf{x}_i} = \mathbf{x}_i - \mu_{\mathbf{x}_i}\tilde{\mathbf{x}}_i$,

$$\begin{aligned} p(\mathbf{q}, \mathbf{x}_i) &= (\mu_\mathbf{q}\tilde{\mathbf{q}} + \mathbf{r}_\mathbf{q})^\top(\mu_{\mathbf{x}_i}\tilde{\mathbf{x}}_i + \mathbf{r}_{\mathbf{x}_i}) \\ &= \mu_\mathbf{q}\mu_{\mathbf{x}_i}p(\tilde{\mathbf{q}}, \tilde{\mathbf{x}}_i) + \mu_\mathbf{q}p(\tilde{\mathbf{q}}, \mathbf{r}_{\mathbf{x}_i}) + \mu_{\mathbf{x}_i}p(\tilde{\mathbf{x}}_i, \mathbf{r}_\mathbf{q}) + p(\mathbf{r}_\mathbf{q}, \mathbf{r}_{\mathbf{x}_i}). \end{aligned}$$

Since we have $p(\tilde{\mathbf{q}}, \mathbf{r}_{\mathbf{x}_i}) \le \|\tilde{\mathbf{q}}\|\|\mathbf{r}_{\mathbf{x}_i}\|$, $p(\tilde{\mathbf{x}}_i, \mathbf{r}_\mathbf{q}) \le \|\tilde{\mathbf{x}}_i\|\|\mathbf{r}_\mathbf{q}\|$, and $p(\mathbf{r}_\mathbf{q}, \mathbf{r}_{\mathbf{x}_i}) \le \|\mathbf{r}_\mathbf{q}\|\|\mathbf{r}_{\mathbf{x}_i}\|$, the following equation holds:

$$p(\mathbf{q}, \mathbf{x}_i) \le \mu_\mathbf{q}\mu_{\mathbf{x}_i}p(\tilde{\mathbf{q}}, \tilde{\mathbf{x}}_i) + \mu_\mathbf{q}\|\tilde{\mathbf{q}}\|\|\mathbf{r}_{\mathbf{x}_i}\| + \mu_{\mathbf{x}_i}\|\tilde{\mathbf{x}}_i\|\|\mathbf{r}_\mathbf{q}\| + \|\mathbf{r}_\mathbf{q}\|\|\mathbf{r}_{\mathbf{x}_i}\|,$$

which completes the proof. □

C Complexity of each approach

Table 7 shows the time and space complexity of each approach. As shown in this table, our approach is more efficient than the other score-aware quantization approaches both in terms of computational and spatial complexity.

D Recall_1@100 of each approach

Table 8 shows the Recall_1@100 values for each approach. As shown in the table, our approach achieves higher search accuracy than the other approaches, even when evaluated using Recall_1@100, since it uses scaled codewords to improve accuracy.